

NEXUS AI

Application Flow & Security Architecture

Primary Problem Statement: Demand Forecasting and Order Fulfillment

Status: Hackathon MVP / In Development **Version:** 0.1.0

Predict → Detect → Recommend → Monitor

Purpose: NEXUS AI is a web-based enterprise decision-support platform that uses historical sales, inventory, supplier, and order data to forecast future demand, identify inventory and fulfillment risks, recommend replenishment actions and suitable suppliers, and monitor orders through fulfillment.

1. System Overview

NEXUS AI is a web-based enterprise decision-support platform focused on demand forecasting and order fulfillment. It converts historical sales, inventory, supplier, and order data into forward-looking demand forecasts, inventory-risk signals, procurement recommendations, and fulfillment alerts.

Target Users

- Demand Planners
- Inventory Managers
- Procurement Managers
- Operations Managers
- Business Approvers
- System Administrators

Core Features

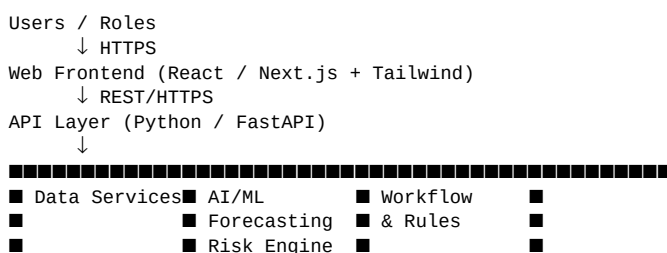
- Demand Forecasting
- Inventory Risk Detection
- Purchase & Vendor Recommendations
- Order/Fulfillment Tracking
- AI Business Assistant
- Command Center Dashboard

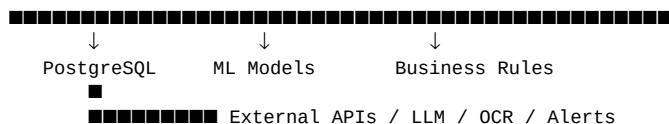
Architectural Principles

- Separation of concerns
- API-first backend architecture
- Role-based access control
- Human-in-the-loop decision making
- Explainable AI recommendations
- Secure-by-default configuration
- Auditability of consequential actions
- Stateless application services where practical
- Replaceable AI/ML components
- Fail-safe behavior for external dependencies

2. Architecture & Tech Stack

2.1 High-Level Architecture





2.2 Frontend

Technology: React / Next.js, Tailwind CSS, and [Charting Library]. The frontend handles authentication UI, dashboards, forecasts, inventory risk, supplier comparison, purchase recommendations, order tracking, alerts, and the AI Assistant. It must never directly access PostgreSQL or internal services.

2.3 Backend

Technology: Python + FastAPI + REST APIs. The backend provides authentication integration, authorization, business logic, data validation, forecast orchestration, risk calculations, procurement recommendations, vendor scoring, order management, audit logging, and AI-service orchestration.

2.4 Database

PostgreSQL is the primary transactional database.

User
 Role
 Product
 Category
 Supplier
 SupplierPerformance
 SalesRecord
 InventorySnapshot
 CustomerOrder
 OrderLine
 Shipment
 FulfillmentEvent
 Forecast
 ForecastMetric
 RiskEvent
 PurchaseRecommendation
 PurchaseRequest
 ApprovalStep
 Alert
 AuditEvent
 Document

2.5 AI / ML Layer

- Demand Forecasting: historical sales, product information and time-based features → future demand.
- Risk Engine: current inventory + forecast demand + lead time + safety stock → risk status.
- Recommendation Engine: forecast + inventory + lead time + safety stock + supplier data → recommended action.
- AI-generated recommendations must not be treated as guaranteed facts.

2.6 Third-Party Integrations

Integration	Purpose
[LLM Provider]	AI Assistant / explanations
[OCR Provider]	Document extraction
[Identity Provider]	Authentication
[Notification Provider]	Alerts
[ERP / API]	Enterprise data integration
[Cloud Provider]	Hosting / infrastructure

3. User Flow / Journey

3.1 Login

User → Login Page → Identity Provider → Authentication
→ Backend Authorization → Role Determination → Command Center

3.2 Command Center

The Command Center is the primary landing page. It surfaces demand forecasts, inventory health, stockout risks, purchase recommendations, supplier performance, order status, delayed shipments, and critical alerts.

3.3 Demand Forecasting

Historical Sales → Validation → Preprocessing → Feature Engineering → Forecast Model → Future Demand → Dashboard

3.4 Inventory Risk

Current Inventory + Forecast Demand + Lead Time + Safety Stock
↓
Risk Engine
↓
Healthy / At Risk / Stockout / Overstock

3.5 Procurement Recommendation

Risk Detected → Calculate Required Inventory → Calculate Recommended Purchase
→ Evaluate Suppliers → Rank Suppliers → Display Recommendation → Accept / Modify / Reject

3.6 Order Fulfillment

CREATED → PROCESSING → SHIPPED → IN_TRANSIT → DELIVERED
■
■ → DELAYED / CANCELLED / FAILED

The system compares expected and actual fulfillment milestones and creates an alert when an order becomes delayed.

3.7 AI Assistant

User Question → AI Assistant → Intent / Query Interpretation → Permission Check
→ NEXUS Backend APIs → Database / ML Results → Context Assembly → LLM → Natural-Language Response

The AI Assistant must be grounded in NEXUS AI data and must not independently invent business facts.

4. Visual Data Flow — Mermaid.js

```
graph TD
    U[User] --> FE[React / Next.js Frontend]
    FE --> AUTH[Authentication Provider]
    AUTH --> API[FastAPI API Layer]
    API --> RBAC[Authorization / RBAC]
    RBAC --> ROUTER[Request Type]
    ROUTER --> DATA[Data Services]
    ROUTER --> FORECAST[Forecast Service]
    ROUTER --> RISK[Risk Engine]
    ROUTER --> REC[Recommendation Engine]
    ROUTER --> ORDER[Order / Fulfillment Service]
    ROUTER --> AI[AI Assistant Service]
    DATA --> DB[(PostgreSQL)]
    FORECAST --> DB
    FORECAST --> ML[ML Forecasting Model]
    ML --> FORECAST
    RISK --> DB
    RISK --> FORECAST
    REC --> RISK
    REC --> SUPPLIER[Supplier Data]
    SUPPLIER --> DB
    ORDER --> DB
    ORDER --> ALERT[Alert Service]
    AI --> AUTHZ[Permission Check]
    AUTHZ --> DATA
    AUTHZ --> FORECAST
    AUTHZ --> RISK
    AUTHZ --> REC
```

```
AI --> LLM[LLM Provider]
API --> AUDIT[Audit Logger]
AUDIT --> DB
DB --> API
API --> FE
```

5. Security & Compliance Architecture

5.1 Security Model

User → Authentication → Authorization → API Validation → Business Rules → Data Access Controls → Audit Logging

5.2 Authentication & Authorization

- Recommended authentication: OAuth 2.0 / OpenID Connect through a managed identity provider.
- Use short-lived access tokens and a secure refresh mechanism.
- Use Role-Based Access Control (RBAC).
- Authorization must be enforced server-side; frontend visibility is not a security control.

Role	Forecast	Procurement	Approvals	Orders	Admin
Planner	Yes	View	No	View	No
Procurement	Yes	Yes	View	Yes	No
Operations	View	View	No	Yes	No
Approver	View	View	Yes	View	No
Admin	Yes	Yes	Yes	Yes	Yes

5.3 Data Protection

- Production traffic must use HTTPS/TLS 1.2+.
- Use encryption at rest for PostgreSQL, object storage, and backups where supported.
- Never store API keys, passwords, JWT secrets, database credentials, or cloud credentials in source code.
- Use environment variables locally and a cloud secret manager in production.

5.4 API Security

- Validate all input types, ranges, identifiers, dates, and enum values.
- Use parameterized SQL / ORM mechanisms to prevent SQL injection.
- Rate-limit authentication, AI, forecasting, expensive, and public endpoints.
- Restrict CORS to trusted origins.
- Consider CSP, HSTS, X-Content-Type-Options, Referrer-Policy, and Permissions-Policy.
- Apply server-side authorization to every protected operation.

5.5 Session & Token Lifecycle

- Use short-lived access tokens; target lifetime: [15–30 minutes], subject to the chosen identity provider.
- Rotate and revoke refresh tokens where used.
- Avoid storing long-lived sensitive tokens in localStorage when a safer managed-cookie architecture is available.
- Logout should invalidate/revoke the session where supported and clear application state.

6. Error Handling & Edge Cases

6.1 Authentication / Authorization

Invalid credentials → 401 Unauthorized
Insufficient permission → 403 Forbidden
Unknown resource → 404 Not Found
Invalid input → 400 Bad Request

Error messages must not reveal database credentials, stack traces, internal service details, or sensitive business information.

6.2 ML Failure

If forecasting fails, log the failure and return a safe error. The rest of the dashboard should remain usable where possible.

6.3 LLM Failure

The AI Assistant is an optional layer. If the LLM provider times out or fails, the core application must continue functioning.

6.4 Database Failure

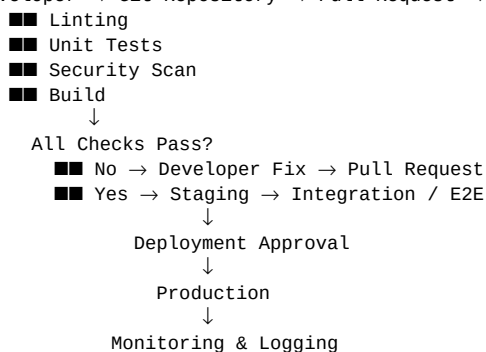
- Return a generic service-unavailable response.
- Log the incident.
- Avoid partial writes through transactions where required.
- Retry only transient failures.
- Use idempotency where duplicate operations are possible.

6.5 External API Failure

- Timeouts
- Retries with backoff
- Circuit-breaker behavior where justified
- Idempotency keys for write operations

7. CI/CD & Deployment Flow

Developer → Git Repository → Pull Request → CI Pipeline



Deployment Environments

Development → Staging → Production

- Development: local implementation and feature work.
- Staging: production-like validation, security testing, model validation, and demo rehearsal.
- Production: authorized deployment of NEXUS AI.

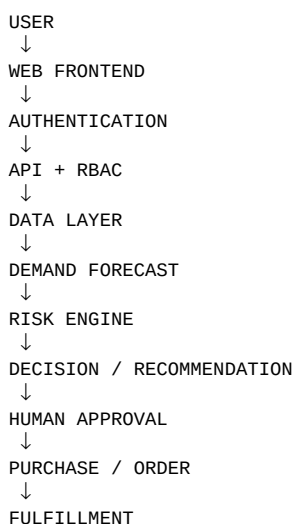
Observability

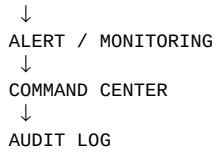
- Application: request count, latency, error rate, authentication failures, API usage.
- AI/ML: forecast generation time, forecast error, recommendation count, recommendation acceptance, AI Assistant failures.
- Infrastructure: CPU, memory, database connections, storage, and network.
- Audit: who, what, when, resource, action, result, and model/rule version.

8. Deployment Readiness Checklist

- Core end-to-end user journey works.
- Demand forecasting works.
- Inventory risk detection works.
- Purchase recommendations work.
- Vendor ranking works.
- Fulfillment tracking works.
- Alerts work.
- AI Assistant is grounded in application data.
- Authentication and RBAC are tested.
- HTTPS is enabled.
- Secrets are not present in the repository.
- CORS is restricted.
- Rate limiting is configured.
- Input validation is implemented.
- Security/dependency scans pass.
- Audit logging is enabled.
- API, ML, LLM, database, and external integration failures are handled.
- Production environment variables and database migrations are verified.
- Production smoke test passes.

9. Recommended Final Request Lifecycle





10. Architectural Boundary

NEXUS AI should maintain a strict distinction between prediction, decision, execution, and monitoring.

Prediction → Risk → Recommendation → Human Decision → Execution → Outcome → Evaluation

The system must not allow an LLM or ML model to directly execute consequential business actions without passing through application authorization and workflow controls.

11. Future Architecture Extensions

- Automated approval workflows
- Invoice / purchase-order document intelligence
- Multi-location inventory optimization
- Advanced anomaly detection
- External ERP integrations
- Scenario simulation
- Automated notifications
- Model retraining pipelines
- Multi-tenant architecture
- Advanced supplier risk intelligence

12. Final Architecture Principle

NEXUS AI should be built as a secure decision-support system, not an autonomous purchasing system.

DATA → AI/ML PREDICTION → RISK ANALYSIS → EXPLAINABLE RECOMMENDATION
→ HUMAN APPROVAL → BUSINESS ACTION → FULFILLMENT MONITORING → AUDIT + FEEDBACK